Field Hours

CSCE 641 Computer Graphics

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A triangle and its weights

You store a solid as triangles because a triangle is planar, so a rasterizer fills it with one rule. Take vertices at (0, 0, 0), (2, 0, 0), and (1, 2, 0). Those 3 points define a face. Any interior point is a mix of the vertices with weights that add to 1. You keep every weight nonnegative so the point stays inside the triangle and not past an edge.

You also store a normal per vertex so later shading can vary across the face. For this flat triangle the 3 normals can match the face normal, which points along z if the face lies in the xy plane. A shared vertex between two faces keeps one position and may keep two normals when a crease should stay sharp. You do not average those normals, or the crease becomes a smooth bend.

One triangle record. Positions can be shared. Normals split only at a crease.

Face as three indices

Three vertex positions

Normals, one or two

Weights inside the sample triangle

1. The centroid uses weights 1/3, 1/3, and 1/3. They add to 1 and stay nonnegative, so the point is inside.

2. The midpoint of the first edge uses weights 1/2, 1/2, and 0. A zero weight puts the point on the boundary, which you still accept.

3. Weights 2, -1, and 0 add to 1, but a negative weight lands outside the triangle. You reject that mix.

Problem 1. A point uses weights 0.2, 0.2, and 0.6. Is it inside the triangle?

Answer. Yes. The three weights are nonnegative and add to 1, so the point is a mix of the three vertices and lies inside the face.

Problem 2. Two faces share a vertex and you want a sharp crease. How many normals do you store there?

Answer. Two. One position is shared. Averaging the two face normals would smooth the crease, so you keep a separate normal for each face.

Lecture 2 · 90 minutes

A camera at (0, 0, 4)

You place a camera at (0, 0, 4) looking toward the origin, with up along y. The view transform moves the world so the camera sits at the origin and looks down the negative z axis. Your sample triangle, which lived in z = 0, then sits 4 units in front of the camera. You build the transform from the camera position, the look direction, and the up vector, three vectors that must stay orthogonal.

You then apply a perspective divide. A point with eye-space depth 4 maps to a smaller image than the same point at depth 2, because you divide x and y by a depth term. With a vertical field of 90 degrees, a point at the edge of that field lands on the top or bottom of the image. You clip before the divide so a vertex behind the camera does not flip across the image.

From a world point to a pixel. Clip before you divide by depth.

World position -> View transform -> Clip -> Perspective divide -> Pixel

A point on the optical axis

1. The origin is 4 units in front of the camera at (0, 0, 4). After the view transform it lies on the optical axis.

2. Its x and y are already 0, so the perspective divide leaves it at the center of the image.

3. A second point halfway to the origin is only 2 units in front. It also lands at the center, and the depth test will treat it as closer.

Problem 1. The camera is at (0, 0, 4) and looks at the origin. How far in front of the camera is a vertex at the origin?

Answer. 4 units. The view transform places that vertex 4 units in front of the camera, on the optical axis.

Problem 2. Why do you clip a vertex that is behind the camera before the perspective divide?

Answer. Dividing by a depth term that has changed sign flips the vertex across the image. Clipping removes it so that flip never happens.

Lecture 3 · 90 minutes

Pixels and a depth buffer

You turn the projected triangle into pixels by walking its screen bounds and testing each pixel against the 3 edges. A pixel is inside when it lies on the same side of each edge as the opposite vertex. On an 8 by 8 image the triangle may cover only a handful of those 64 pixels. You compute barycentric weights per pixel so you can interpolate depth and the vertex attributes without a second pass.

You resolve visibility with a depth buffer, one depth per pixel. When two fragments land on the same pixel, you keep the one with the smaller distance and discard the other. If the near fragment has depth 2 and the far fragment has depth 5, the buffer stores 2 and the color of the near fragment. You write depth and color together so a later fragment cannot paint over a closer surface.

A pixel is colored only if it is inside the triangle and closer than the stored depth.

1. Test the three edges

2. Interpolate the depth

3. Compare with the buffer

4. Write color and depth

Two fragments, one pixel

1. Fragment A at a pixel has depth 2. The buffer is empty, so you write A's color and store 2.

2. Fragment B at the same pixel has depth 5. Because 5 is larger than 2, you discard B and leave both color and depth.

3. A later fragment with depth 1 replaces the color and the stored depth. The buffer now holds 1.

Problem 1. The depth buffer holds 3 at a pixel. A new fragment has depth 3.5. Do you replace the color?

Answer. No. 3.5 is farther than 3, so you discard the new fragment and leave the stored color and depth.

Problem 2. A pixel has barycentric weights 0.5, 0.5, and 0.2. Is the pixel inside the triangle?

Answer. No. The weights must add to 1 and stay nonnegative. These add to 1.2, so the pixel fails the inside test.

Lecture 4 · 90 minutes

Diffuse light and a highlight

You shade a fragment with a diffuse product: light intensity times the diffuse coefficient times the cosine of the angle between the normal and the light. If that cosine is 0, the surface faces the light edge-on and the diffuse term is 0. You clamp a negative cosine to 0 so a face that points away from the light does not subtract light it never received. The product is evaluated at the fragment, using the interpolated normal.

You add a specular lobe that depends on the view. With the light and the eye both 45 degrees off the normal in one plane, the reflection of the light heads toward the eye and the highlight is strong. Move the eye by another 20 degrees and the same reflection misses the eye, so the highlight drops. You evaluate this per fragment. Once per triangle, the highlight stays a flat blotch.

Diffuse light uses the normal and the lamp. The highlight also uses the eye.

Diffuse term

Normal and light

Clamp the cosine at 0

No view direction

Specular lobe

Reflected light direction

Strong at 45 degrees

Computed per fragment

A light along the normal

1. The normal is (0, 0, 1) and the light direction is (0, 0, 1). The cosine is 1.

2. Intensity 2 times diffuse coefficient 0.5 times cosine 1 gives a diffuse brightness of 1.

3. A second light from the side has cosine 0, so its diffuse contribution is 0 and you add nothing.

Problem 1. Intensity is 4, the diffuse coefficient is 0.25, and the cosine is 1. What is the diffuse brightness?

Answer. 1. You multiply 4 by 0.25 by 1.

Problem 2. The cosine between the normal and the light is -0.3. What diffuse contribution do you add?

Answer. 0. You clamp a negative cosine to 0 because that face does not face the light.

Lecture 5 · 90 minutes

Rays, shadows, and a bounce cap

You trace a ray from the camera at (0, 0, 4) through one pixel and keep the closest hit. The ray starts at the camera and aims at that pixel on a plane at z = 3, one unit ahead. You test each triangle and keep the smallest positive distance. You ignore a hit at distance 0 or less so the camera is not treated as a surface.

You spawn a shadow ray from the hit toward a point light. If anything intersects that ray before the light, you add no direct light at the hit. A reflection ray leaves along the mirror direction. With a recursion cap of 2, you follow one more bounce and then stop, so mirrors cannot loop. You shade the closest hit with the cosine rule, then add the reflection term.

One pixel starts one camera ray, then a short chain of secondary rays.

1. Cast the camera ray

2. Keep the closest hit

3. Cast a shadow ray

4. Reflect, then stop at 2

A hit 3 units out

1. The camera ray starts at (0, 0, 4) and hits a triangle at distance 3. A second triangle at distance 6 loses.

2. The shadow ray from the hit reaches the light with no intersection, so you add the direct cosine term.

3. The reflection ray is cast once. The cap of 2 means the reflected hit does not spawn its own reflection.

Problem 1. Two triangles sit at distances 3 and 6 along the same camera ray. Which hit do you shade?

Answer. The triangle at distance 3. You keep the smallest positive distance and ignore the farther hit.

Problem 2. A reflection path wants a third bounce, but the cap is 2. What do you do?

Answer. Stop. You already used the two allowed bounces, so you do not spawn another reflection ray.

Lecture 6 · 90 minutes

Patches and radiosity

You switch from rays to patches when the light is mostly diffuse. Split a room into 20 patches. Each patch has an area and a view of the others. The form factor from patch i to patch j is the fraction of light leaving i that arrives at j. You compute those fractions once, because the mechanism depends on geometry, not on how bright the patches currently are.

You then solve for radiosities. A patch radiosity is its emitted light plus its reflectance times the light gathered from the other 19 patches, each weighted by a form factor. If one patch emits 1 and the rest emit 0, light still reaches a facing patch, and that patch reflects a fraction. You iterate the gather until the change in every patch drops below 0.01.

Form factors come from geometry. Radiosity comes from repeating the gather.

Split into 20 patches -> Store form factors -> Gather the light -> Stop below 0.01

Two patches facing each other

1. Patch A emits 1 and has reflectance 0. Patch B emits 0 and has reflectance 0.5.

2. The form factor from B to A is 1, a toy case where all of the light B receives comes from A.

3. B radiosity is 0 plus 0.5 times 1, which is 0.5. A stays at 1 because its reflectance is 0.

Problem 1. You have 20 patches. How many form factors do you store if every patch has a factor to every patch, including a zero to itself?

Answer. 400. Each of the 20 patches stores a factor to each of the 20 patches, and 20 times 20 is 400.

Problem 2. Patch B has reflectance 0.5, emission 0, and gathers 1 from the other patches. What is its radiosity?

Answer. 0.5. Radiosity is emission plus reflectance times gathered light, so 0 plus 0.5 times 1.

Lecture 7 · 90 minutes

A matrix per frame

You animate the triangle by changing a transform over time, not by rewriting the 3 vertex positions by hand each frame. At frame 0 the triangle sits in its rest pose. At frame 30, half a second later at 60 frames per second, a rotation about y has turned it. You store the rest positions in a buffer and send a new matrix each frame. The vertices stay in that buffer.

You hand the rest of the frame to the graphics processor. It runs a vertex stage on each of the 3 vertices, then a raster stage that builds fragments, then a fragment stage that shades them. The depth test still happens, in fixed

hardware beside the color write. You keep the vertex buffer static and the matrix in a small uniform slot so the mesh is not reloaded when only the pose changed.

The mesh stays in the buffer. The matrix changes with the frame number.

Rest vertex buffer -> Vertex stage -> Raster stage -> Fragment stage

A 60 frame spin

1. The mesh buffer holds the 3 rest vertices for all 60 frames. You upload that buffer once.

2. Frame 0 uses a matrix with no rotation. Frame 30 uses a half turn about y, which is 180 degrees.

3. Only the uniform matrix changes between those frames. You do not upload the 3 vertices again.

Problem 1. The animation is 60 frames and the pose changes every frame. How many times do you upload the 3 rest vertices?

Answer. Once. You keep them in a static buffer and upload a new matrix each frame instead.

Problem 2. Name the three processor stages that run after you submit the mesh, in order.

Answer. Vertex stage, then raster stage, then fragment stage. The depth test sits with the color write, not as a fourth stage you schedule.

Lecture 8 · 90 minutes

Choosing how a frame is made

You can now take the 3-vertex triangle, put a camera at (0, 0, 4), and produce a shaded image either by rasterizing with a depth buffer or by tracing one ray per pixel. You can also bounce diffuse light across 20 patches, and you can move the triangle over 60 frames by changing a matrix. The practical split is a raster path for interactive frames and a ray path when you need a reflection with a cap of 2.

You still match the mechanism to the path the light takes. A depth buffer can hide a triangle at depth 5, but it does not tell you if that point sees a lamp. A shadow ray does. Radiosity will not give a sharp mirror, because a form factor spreads light by area, not along 1 reflection direction. You pick the tool that matches the scene.

Two ways to hide what should not be seen, and the question each one answers.

Depth buffer

Compares two depths

Keeps the closer color

No lamp visibility

Shadow ray

From the hit to the lamp

A hit means shadow

Drops the direct term

One triangle, two tests

1. Rasterize the triangle from the camera at (0, 0, 4). The depth buffer stores the near depth at each covered pixel.

2. Trace one camera ray through the center pixel. It hits at distance 4 if the triangle contains the origin.

3. From that hit, a shadow ray either reaches the lamp or stops on a blocker. The raster image has no such test unless you add one.

Problem 1. Combine the depth test and a shadow ray. A pixel has a far fragment at depth 5 and a near fragment at depth 2. The near hit shadow ray meets a blocker. What direct light do you write?

Answer. None. You keep the near fragment because depth 2 is closer than depth 5, and the shadow ray says that hit is blocked, so the direct term is 0.

Problem 2. You need a sharp mirror and a 60 frame animation. Which two mechanisms do you use?

Answer. A reflection ray under the cap of 2, and a matrix that changes each frame while the 3 rest vertices stay in a static buffer. Radiosity would blur the mirror.